Mandelbrot-halmaz generálása OpenCL használatával

Rövid leírás

– A feladat célja egy Mandelbrot-halmazt kirajzoló program készítése volt a tárgyban tanult párhuzamos eszközök használatával. A program C nyelven készült, és OpenCL segítségével GPU-n végzi a számítást.

– A felhasználó megadja a kép szélességét, magasságát és a maximális iterációszámot. A program ezután lefuttat egy OpenCL kernelt, amelyben minden work-item egy pixelhez tartozó értéket számol ki. Az eredmény egy PPM képfájlba kerül mentésre.

– A Mandelbrot-halmaz számítása jól párhuzamosítható, mert az egyes pixelek számítása egymástól független. Emiatt a feladat alkalmas GPU-s végrehajtásra.

– A program méri a kernel futási idejét, kezeli a hibás vagy túl nagy bemeneteket, és a kimeneti fájlokat egyedi néven menti el.

Mérési környezet

– A méréseket saját gépen végeztem. A program képes GPU és CPU módon is futni, ezért a dokumentumban az eredeti GPU-alapú mérések mellett külön összehasonlító elemzés is szerepel.

– A frissített összehasonlító mérésben használt eszközök: NVIDIA GeForce GTX 1650, Intel(R) Iris(R) Xe Graphics, valamint CPU sequential mód.

– A futási idő mérése OpenCL profiling segítségével történt, ezért a megadott értékek a kernel végrehajtási idejét jelentik milliszekundumban.

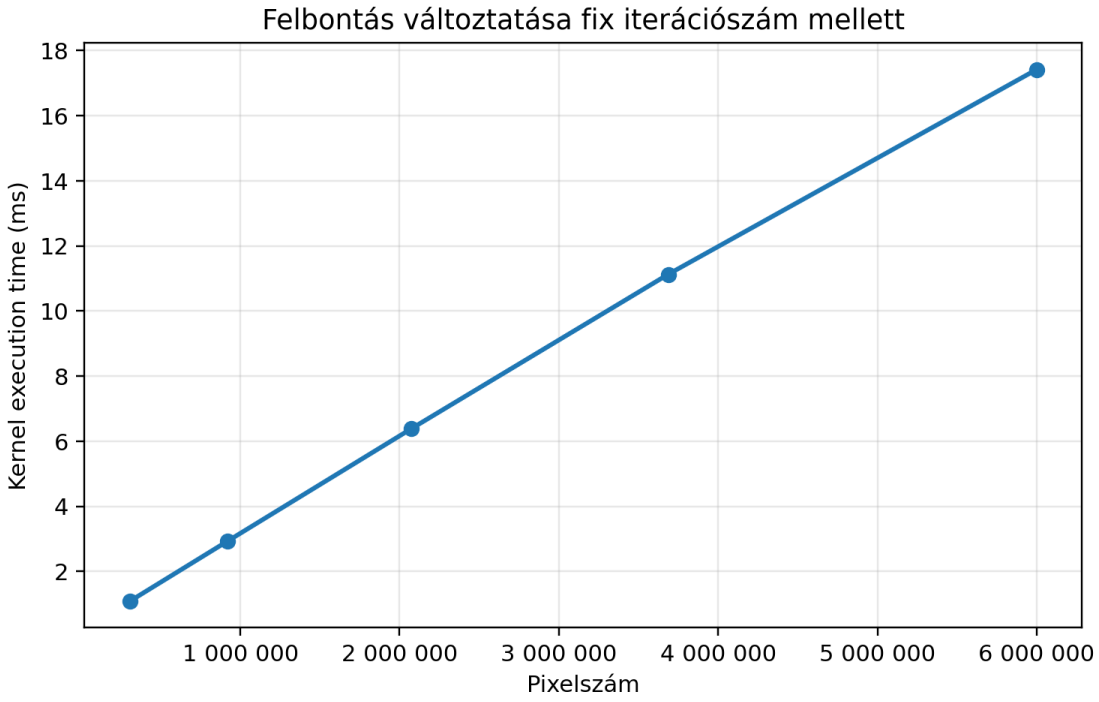
– A cél annak bemutatása, hogy a probléma méretének, az iterációszámnak és a választott számítási eszköznek a változása hogyan hat a futási időre.

1. mérés: Felbontás változtatása fix iterációszám mellett (integrált GPU)

Ebben a mérésben az iterációszám értéke állandó volt, és csak a felbontást változtattam. A kapott eredmények az alábbi táblázatban láthatók.

Szélesség	Magasság	Pixelszám	Iterációszám	Kernel execution time (ms)
640	480	307 200	1000	1.0885
1280	720	921 600	1000	2.9365
1920	1080	2 073 600	1000	6.3847
2560	1440	3 686 400	1000	11.1260
3000	2000	6 000 000	1000	17.4172

1. táblázat – Felbontás és futási idő kapcsolata fix iterációszám mellett



1. ábra – A felbontás növekedésének hatása a futási időre

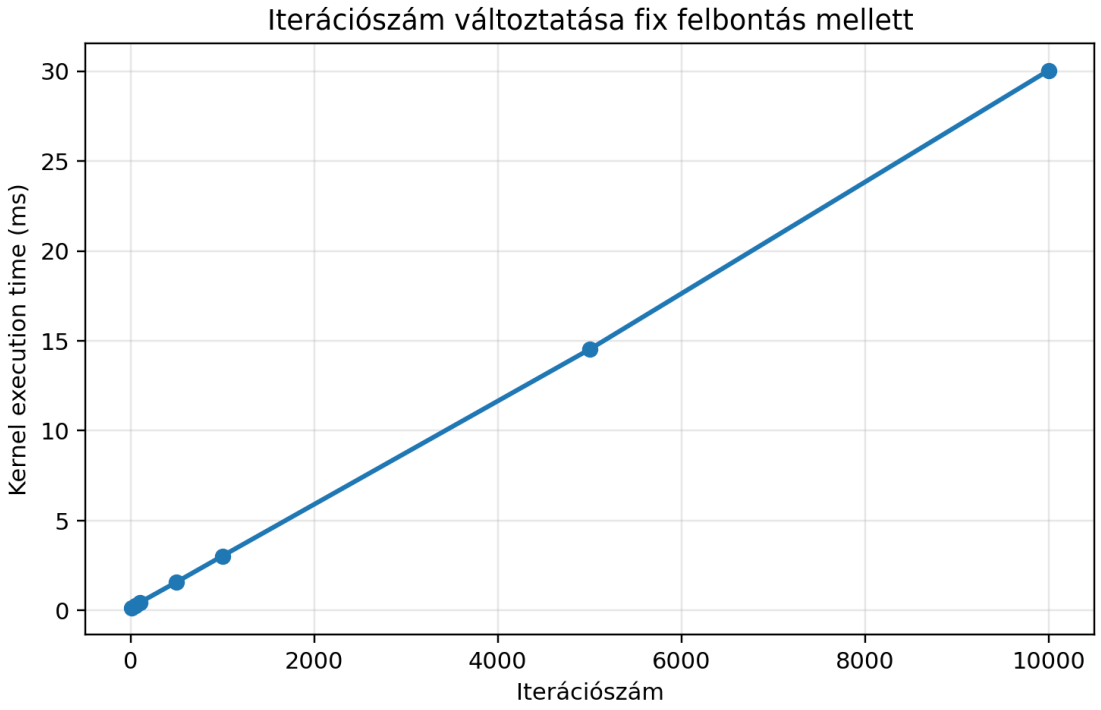
Rövid kiértékelés: A mérési eredmények alapján jól látható, hogy a felbontás növelésével a futási idő is növekszik. Ennek oka, hogy nagyobb felbontás esetén több pixelhez kell külön számítást végezni, ezért nő a feldolgozandó adatok mennyisége.

2. mérés: Iterációszám változtatása fix felbontás mellett (integrált GPU)

Ebben a mérésben a felbontás értéke állandó volt, és az iterációszámot változtattam. A kapott eredmények az alábbi táblázatban láthatók.

Szélesség	Magasság	Pixelszám	Iterációszám	Kernel execution time (ms)
1000	1000	1 000 000	10	0.1439
1000	1000	1 000 000	50	0.2720
1000	1000	1 000 000	100	0.4449
1000	1000	1 000 000	500	1.5827
1000	1000	1 000 000	1000	3.0337
1000	1000	1 000 000	5000	14.5293
1000	1000	1 000 000	10000	30.0470

2. táblázat – Iterációszám és futási idő kapcsolata fix felbontás mellett



2. ábra – Az iterációszám növekedésének hatása a futási időre

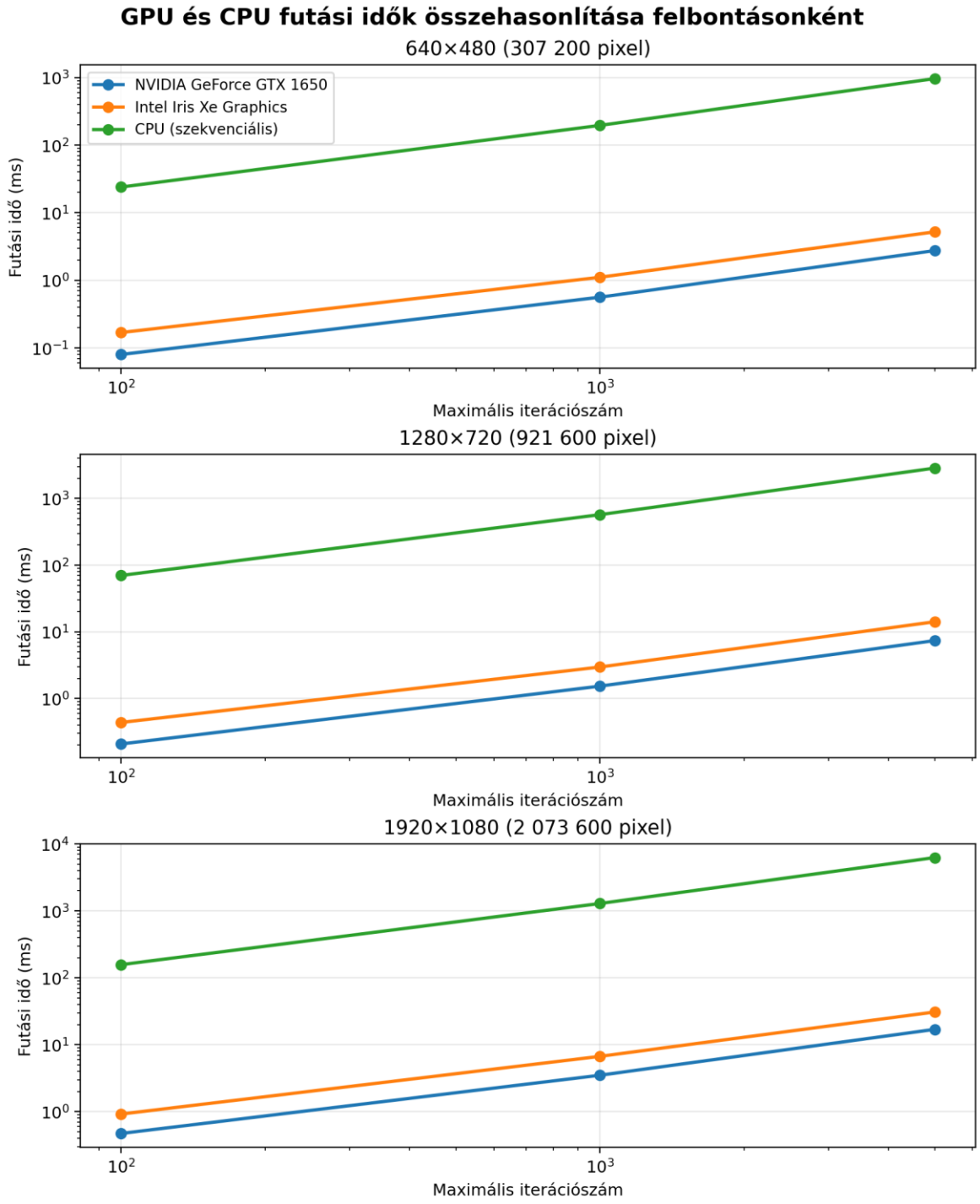
Rövid kiértékelés: A mérési eredmények alapján megfigyelhető, hogy az iterációszám növelésével a futási idő is emelkedik. Ennek oka, hogy egy pixel feldolgozásához több számítási lépés szükséges, ha magasabb maximális iterációszámot adunk meg.

3. mérés: Eszközök összehasonlítása (két GPU és CPU)

Ebben a kiegészítő mérésben ugyanazokat a bemeneti kombinációkat háromféle végrehajtási módban hasonlítottam össze: NVIDIA GeForce GTX 1650, Intel Iris Xe Graphics és CPU sequential. A cél annak bemutatása volt, hogyan alakul a futási idő a két különböző videokártyán, illetve mekkora különbség látható a GPU-s és CPU-s végrehajtás között.

Szélesség	Magasság	Pixelszám	Iterációszám	GTX 1650 (ms)	Intel Iris Xe (ms)	CPU (ms)
640	480	307 200	100	0.080	0.169	24.000
640	480	307 200	1000	0.563	1.110	196.000
640	480	307 200	5000	2.752	5.211	965.000
1280	720	921 600	100	0.208	0.437	70.000
1280	720	921 600	1000	1.532	2.966	571.000
1280	720	921 600	5000	7.378	14.138	2851.000
1920	1080	2 073 600	100	0.471	0.918	157.000
1920	1080	2 073 600	1000	3.494	6.693	1289.000
1920	1080	2 073 600	5000	16.942	30.813	6258.000

3. táblázat – Azonos bemenetek futási ideje a három végrehajtási módon



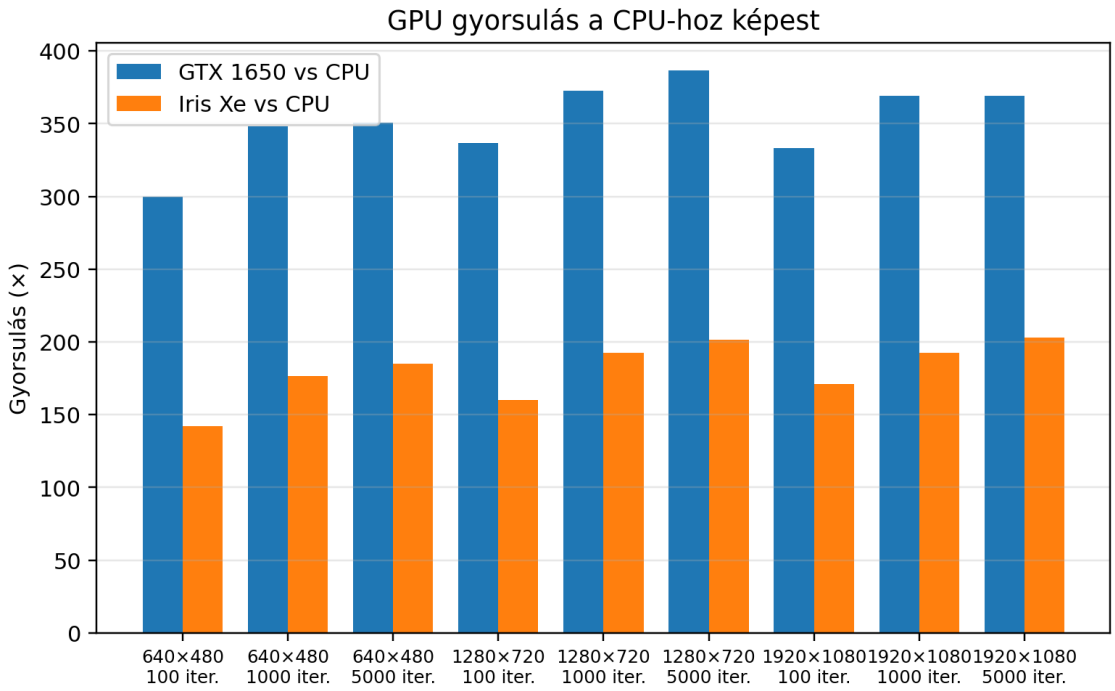
3. ábra – GPU és CPU futási idők összehasonlítása felbontásonként

Rövid kiértékelés:

Minden vizsgált felbontás és iterációszám mellett a GTX 1650 adta a legjobb eredményt, utána az Intel Iris Xe következett, míg a CPU-szekvenciális futás nagyságrendekkel lassabb maradt. A logaritmikus skála jól megmutatja, hogy a különbség nem csak kis, hanem nagyobb problémaméretek mellett is stabilan megmarad.

Eszköz	Átlagos idő (ms)	Minimum (ms)	Maximum (ms)	Átlagos gyorsulás a CPU-hoz képest
NVIDIA GeForce GTX 1650	3.713	0.080	16.942	351.79×
Intel Iris Xe Graphics	6.939	0.169	30.813	180.54×
CPU (szekvenciális)	1375.667	24.000	6258.000	1.00×

4. táblázat – Összesített statisztika a három végrehajtási módról



5. ábra – A GPU-k gyorsulása a CPU-hoz képest

Rövid kiértékelés:

A két GPU közül a GTX 1650 minden mérésben gyorsabb volt az Intel Iris Xe-nél; az előnye jellemzően 1,82× és 2,11× között alakult, az átlagos gyorsulás pedig 1,96× volt. A CPU-hoz viszonyítva még látványosabb a különbség: a GTX 1650 átlagosan 351,79×, az Intel Iris Xe pedig 180,54× gyorsabbnak bizonyult a vizsgált esetekben.

Összegzés

– A projekt során egy OpenCL-alapú, párhuzamosított Mandelbrot-generáló program készült, amely jól demonstrálja, hogyan használható ki a GPU az egymástól független pixelenkénti számítások gyorsítására.

– Az eredeti mérések igazolták, hogy mind a felbontás, mind az iterációszám növelése együtt jár a futási idő növekedésével.

– A frissített összehasonlító mérések egyértelműen megmutatták, hogy a dedikált GPU (GTX 1650) teljesít a legjobban, az integrált GPU (Intel Iris Xe) is nagyon erős gyorsulást ad, míg a CPU-szekvenciális végrehajtás lényegesen lassabb marad.

– A dokumentáció így már nemcsak a problémaméret hatását mutatja be, hanem azt is, hogy ugyanazon feladat esetén milyen jelentős különbséget okoz a választott hardver és végrehajtási mód.